# Architectural Foundations of Relational Subqueries, Quantified Predicates, and Virtual Views: Modular Decomposition, Correlated Loop Complexity, NULL-Safe Existence Semantics, and AST Merge Mechanics in SQL Database Systems

**Pavan Gopala**
Database Systems and Relational Architecture Group
pavan@enterprise-sql.org

## Abstract

In relational database management systems (RDBMS), decomposing complex analytical questions into modular, verifiable expressions relies fundamentally on nested subqueries and relational view abstractions. Built on relational calculus and first-order logic, subqueries operationalize the divide-and-conquer paradigm across filtering (`WHERE`, `HAVING`), projection (`SELECT`), and transient relation derivation (`FROM`). This paper presents an exhaustive systems engineering investigation of relational subqueries, correlated execution dynamics, quantified comparison predicates, and virtual views derived from foundational relational theory and enterprise SQL engines (such as MySQL InnoDB). We formalize the cardinality constraints of scalar subqueries, contrasting deterministic singleton reductions against runtime Error 1242 violations. We dissect multi-row set membership (`IN`) and derived tables in the `FROM` clause, proving mathematically why derived tables require mandatory aliasing and how they resolve the ANSI SQL prohibition against nested aggregate functions (e.g., computing `MIN(AVG(psp))`). Furthermore, we formalize quantified comparison operators (`ALL` and `ANY`), establishing the semantic necessity of `ANY` for multi-row inequality evaluations and demonstrating the non-intuitive vacuous truth behavior of `ALL` over empty sets. We conduct rigorous computational complexity analyses contrasting decoupled independent subqueries against correlated subqueries, demonstrating how unnesting and decorrelation reduce execution latency from quadratic $\mathcal{O}(N \cdot M)$ tuple iterations to linear $\mathcal{O}(N+M)$ index lookups. Crucially, we prove the fundamental three-valued logic (3VL) divergence between `NOT IN` and `NOT EXISTS`, revealing how NULL values in subqueries catastrophically yield `UNKNOWN` and empty result sets under `NOT IN`, whereas `NOT EXISTS` guarantees complete NULL-immunity. Finally, we dissect the execution mechanics of relational views, contrasting AST expansion (`ALGORITHM = MERGE`) with internal materialization (`TEMPTABLE`), and establish the relational invariants governing view updatability. All principles are substantiated by vector TikZ architectural diagrams, relational truth tables, formal complexity proofs, and production SQL implementations.

**Keywords:** Relational Databases, Subqueries, Correlated Subqueries, Quantified Predicates, ALL and ANY, EXISTS, Three-Valued Logic, Virtual Views, Query Optimization, MySQL InnoDB.

## 1. Introduction

In modern enterprise computing architectures, Relational Database Management Systems (RDBMS) such as MySQL InnoDB, PostgreSQL, and Oracle are tasked with executing complex analytical and transactional queries over highly normalized relational schemas. While elementary queries retrieve or filter data from single tables, enterprise decision systems frequently require multi-stage relational reasoning: identifying entities that exceed dynamically calculated population thresholds, verifying referential memberships across distinct role hierarchies, computing non-nestable aggregate summaries, and establishing access-controlled data abstractions.

To resolve complex relational inquiries declaratively, SQL provides the **Subquery** mechanism. Formalized within relational calculus and first-order predicate logic [1,2], a subquery (or nested query) is an auxiliary `SELECT` statement encapsulated within an enclosing parent query block. Subqueries embody the classical algorithmic principle of **Divide and Conquer**: decomposing a monolithic relational problem into modular, verifiable subproblems whose intermediate results feed directly into upstream evaluation operators.

Subqueries operate across multiple structural clauses of SQL:

1. **Filtering Clauses (`WHERE` and `HAVING`)**: Enforcing scalar comparisons ($=, <, >$), set-membership evaluations (`IN`), quantified relational bounds (`ALL`, `ANY`), and existential verifications (`EXISTS`).
2. **Table-Expression Clauses (`FROM`)**: Constructing transient derived relations that bypass SQL's prohibition against nested aggregate functions.
3. **Projection Clauses (`SELECT`)**: Computing scalar dimensional mappings dynamically for each output row.

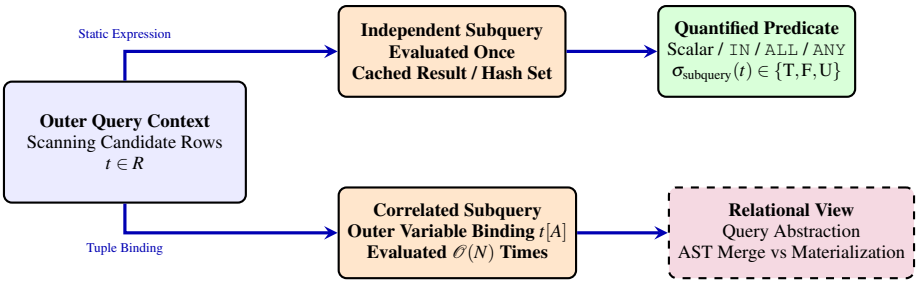



Figure 1: The Architectural Taxonomy of Relational Subqueries and Virtual Views. Subqueries decompose parent relational operations into independent cached evaluations or tuple-correlated iterations, while relational views encapsulate multi-stage logic into reusable schema abstractions.

Based on pedagogical and industrial database engineering curriculum notes (specifically [3]), this research paper presents an exhaustive architectural analysis of relational subqueries, correlated query processing, quantified comparison predicates, and virtual relational views. We formalize the algebraic mechanics of scalar, columnar, and table subqueries, analyze the computational complexity trade-offs of subquery correlation ($\mathcal{O}(N+M)$ vs. $\mathcal{O}(N \cdot M)$), prove the mathematical and NULL-handling divergence between `IN` and `EXISTS`, and examine the physical execution of virtual views under MySQL InnoDB.

## 2. Scalar Subqueries and Cardinality Enforcement

The most elementary class of nested query is the **Scalar Subquery**, which produces an output relation consisting of exactly one row and one column ($1 \times 1$).

### 2.1. Syntax and Operational Mechanics

Consider the canonical `students` relation from [3], structured with attributes `id`, `name`, `batch_id`, and `psp` (problem-solving percentage). Suppose business logic requires identifying all students who performed strictly better than a specific individual (e.g., student with `id = 2`, named Rohit with `psp = 90`):

```
SELECT id, name, batch_id, psp
FROM students
WHERE psp > (
    SELECT psp
    FROM students
    WHERE id = 2
);
```

The execution engine processes this statement in two distinct phases:

1. **Inner Scalar Evaluation**: The subquery `SELECT psp FROM students WHERE id = 2` executes against the primary key index, locating Rohit's record and extracting the scalar value 90.
2. **Outer Selection Execution**: The scalar literal 90 replaces the subquery expression in the outer Abstract Syntax Tree (AST). The outer query evaluates `WHERE psp > 90`, emitting only student Sachin (`id = 7`, `psp = 95`).

### 2.2. Cardinality Invariants and Runtime Violations

Standard relational comparison operators ($=, !=, <, <=, >, >=$) are strictly binary operations defined over scalar operands:

$$\text{op} : D \times D \longrightarrow \{\text{TRUE}, \text{FALSE}, \text{UNKNOWN}\} \quad (1)$$

If an inner subquery evaluated within a scalar comparison operator returns more than one tuple:

```
-- RUNTIME CARDINALITY VIOLATION:
SELECT * FROM students
WHERE psp > (SELECT psp FROM students WHERE batch_id =
    1);
```

Because `batch_id = 1` contains multiple students (Virat with `psp = 80` and Rohit with `psp = 90`), the inner query returns a multi-row column vector $\{80, 90\}$. The execution engine cannot evaluate whether a candidate score is greater than an entire set without explicit quantification, aborting execution with MySQL Error 1242:

```
ERROR 1242 (21000): Subquery returns more than 1 row
```

#### 2.2.1. Defensive Production Engineering Patterns

To guarantee that subqueries evaluated within scalar operators never trigger runtime cardinality exceptions, software engineers deploy two defensive patterns:

- **Deterministic Scalar Aggregators**: Wrapping the inner projection in an aggregate function (such as `MAX(psp)` or `MIN(psp)`), guaranteeing that regardless of row volume, exactly one scalar is emitted.
- **Bounded Deterministic Windowing**: Appending an explicit sort and limit directive: `ORDER BY psp DESC LIMIT 1`.

## 3. Set-Membership Subqueries: The IN Clause and Semi-Joins

When a subquery returns multiple rows along a single attribute column (a 1D column vector of dimension $m \times 1$), standard scalar comparison operators fail. In such scenarios, set-membership operators, primarily `IN`, are deployed.

### 3.1. Problem Formulation: Identifying Dual-Role Entities

Consider an operational table where individuals hold dual classifications: students who also serve as Teaching Assistants (`is_ta = 1`):

```
-- Table Schema
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    is_student TINYINT(1) DEFAULT 1,
    is_ta TINYINT(1) DEFAULT 0
);
```

Requirement: *"Find the student IDs of all individuals whose name matches the name of any Teaching Assistant."*

This query is formulated via an `IN` subquery:

```
SELECT id, name
FROM students
WHERE name IN (
    SELECT name
    FROM students
    WHERE is_ta = 1
);
```

**Definition 1** (Set-Membership Subquery Semantics)**.** *Let $R$ be the outer relation and $S_{sub}$ be the set of values emitted by the inner query:*

$$S_{sub} = \{t_{inner}[A] \mid t_{inner} \in S \wedge p(t_{inner}) = TRUE\} \quad (2)$$

*A candidate outer tuple $t_{outer} \in R$ is retained if and only if:*

$$t_{outer}[A] \in S_{sub} \iff \exists s \in S_{sub} \text{ such that } t_{outer}[A] = s \quad (3)$$

In the pedagogical dataset from [3]:

- TAs are Virat (`id = 1`), Hardik (`id = 4`), and Rohit (`id = 6`).
- The inner query emits $S_{\text{sub}} = \{\text{'Virat'}, \text{'Hardik'}, \text{'Rohit'}\}$.
- In the outer query, student Rohit (`id = 2`, who is not a TA) is correctly captured alongside `id = 1, 4, 6` because his name matches an element in $S_{\text{sub}}$. The query returns IDs: $\{1, 2, 4, 6\}$.

### 3.2. Query Optimizer Semi-Join Transformations

In naive relational engines, an `IN` subquery was executed as a nested loop: for every outer row, the inner subquery was re-evaluated, resulting in $\mathcal{O}(N \cdot M)$ complexity.

Modern query optimizers (including MySQL 8.0+) transform `WHERE col IN (SELECT ...)` into an optimized **Semi-Join**:

$$R \ltimes S = \{r \in R \mid \exists s \in S \text{ such that } r.A = s.A\} \quad (4)$$

Unlike an Inner Join (which duplicates outer rows if multiple inner rows match), a Semi-Join emits an outer row at most once, immediately terminating inner row scans upon the first referential match.

The optimizer selects among four primary semi-join execution strategies:

1. **Materialization**: The inner query is evaluated once, deduplicated, and stored in an in-memory hash table. Outer rows probe the hash table in $\mathcal{O}(1)$ time.
2. **FirstMatch**: The outer table drives a B+ Tree index seek into the inner table, halting immediately when the first match is confirmed.
3. **LooseScan**: The inner table index is scanned using grouping skip-scans, emitting distinct values to join against the outer table.
4. **Duplicate Weedout**: The tables are joined via standard inner join, and a temporary bitmap table removes duplicate outer row IDs.

## 4. Derived Tables in the FROM Clause and Non-Nesting Invariants

When a subquery appears in the `FROM` clause of a parent statement, it is formally termed a **Derived Table** (or inline view).

### 4.1. The Mandatory Alias Invariant

In ANSI SQL and MySQL, derived tables operate as anonymous relational expressions. Because every relation within the `FROM` clause must possess a resolvable namespace in the query data dictionary:

**Every derived table in the `FROM` clause MUST be assigned an explicit table alias.**

Omitting an alias triggers MySQL Error 1248:

```
ERROR 1248 (42000): Every derived table must have its
    own alias
```

### 4.2. The Non-Nesting Aggregate Invariant

A core syntactic and operational constraint of ANSI SQL is the strict prohibition of nested aggregate functions:

**Expressions of the form `MIN(AVG(col))` or `SUM(COUNT(col))` are strictly illegal in SQL.**

Table 1: Comparative Dimensionality and Syntactic Clauses of Relational Subqueries

| Subquery Class | Output Geometry | Permissible Clauses | Evaluation Operators | Optimizer Execution Strategy |
|---|---|---|---|---|
| **Scalar Subquery** | Single Value ($1 \times 1$) | `SELECT`, `WHERE`, `HAVING` | Scalar comparisons ($=, <, >, \leq, \geq$) | Evaluated once; value cached into AST literal. |
| **Columnar Subquery** | Single Column ($m \times 1$) | `WHERE`, `HAVING` | `IN`, `NOT IN`, `ANY`, `ALL` | Semi-join materialization or temporary hash set lookup. |
| **Row Subquery** | Single Row ($1 \times k$) | `WHERE`, `HAVING` | Tuple equality: $(a,b) = (\text{subquery})$ | Evaluated once; tuple unpacked into scalar registers. |
| **Table Subquery** | Multi-Row, Multi-Col ($m \times k$) | `FROM`, `JOIN` | Relational scans, explicit aliasing | Derived table merging or on-disk/in-memory temporary table. |

Why are nested aggregates disallowed? An aggregate function collapses a multi-set of rows into a single scalar summary per group. At the time an aggregate executes within a query block, the input rows have already been collapsed; there is no secondary grouping context within the same logical lifecycle phase to evaluate an outer aggregate.

### 4.3. Multi-Stage Aggregation via Derived Tables

Consider the challenging multi-stage problem from [3]:

*"Find the data of all students whose PSP is strictly greater than the minimum batch average PSP across the entire academy."*

Because `MIN(AVG(psp))` is forbidden, this query requires a three-stage relational decomposition:

1. **Stage 1 (Inner Grouping)**: Compute the average PSP for each batch:

```
SELECT avg(psp) AS avg_psp
FROM students
GROUP BY batch_id;
```

2. **Stage 2 (Derived Table Reduction)**: Calculate the minimum of those batch averages by querying the materialized derived table:

```
SELECT min(avg_psp)
FROM (
    SELECT avg(psp) AS avg_psp
    FROM students
    GROUP BY batch_id
) AS batch_averages;
```

3. **Stage 3 (Outer Selection)**: Filter individual students whose personal PSP exceeds this dynamically computed threshold:

```
SELECT * FROM students
WHERE psp > (
    SELECT min(avg_psp)
    FROM (
        SELECT avg(psp) AS avg_psp
        FROM students
        GROUP BY batch_id
    ) AS batch_averages
);
```

As illustrated in Figure 2, derived tables enable multi-pass analytical reductions while maintaining strict compliance with relational algebra invariants.

## 5. Quantified Comparison Predicates: ALL and ANY

When comparing a scalar attribute against a set of values emitted by a columnar subquery using relational inequalities ($<, \leq, >, \geq$), SQL provides **Quantified Comparison Predicates**: `ALL` and `ANY` (synonymous with `SOME`).

### 5.1. Mathematical Formalization

Let $x$ be a scalar attribute value and $S = \{s_1, s_2, \ldots, s_m\}$ be a non-empty set of values returned by a subquery:

$$x > \text{ALL}(S) \iff \forall s \in S,\ x > s \iff x > \max(S) \tag{5}$$

$$x < \text{ALL}(S) \iff \forall s \in S,\ x < s \iff x < \min(S) \tag{6}$$

$$x > \text{ANY}(S) \iff \exists s \in S,\ x > s \iff x > \min(S) \tag{7}$$

$$x < \text{ANY}(S) \iff \exists s \in S,\ x < s \iff x < \max(S) \tag{8}$$

$$x = \text{ANY}(S) \iff \exists s \in S,\ x = s \iff x \in S \iff x \text{ IN } S \tag{9}$$

### 5.2. Why ANY is Necessary: The Limitation of IN

A common misconception among database developers is attempting to use `IN` with inequality comparisons:

```
-- SYNTAX ERROR: IN only supports equality!
SELECT * FROM students WHERE psp > IN (SELECT ...);
```

The `IN` predicate is mathematically restricted to set equality ($\in$). To express an inequality comparison against an arbitrary set, the query **must** employ `ANY` or `ALL`:

```
-- Correct formulation:
SELECT * FROM students WHERE psp > ANY (SELECT ...);
```

### 5.3. Pedagogical Analysis of ALL and ANY

Consider the domain examples from [3]:

- **Universal Bound (`ALL`)**: *"Virat has scored more than all other players."* Virat's score (80) is compared against $\{90, 78, 89, 70, 78, 95\}$. Because $80 < 90$ and $80 < 95$, the universal condition fails (FALSE).

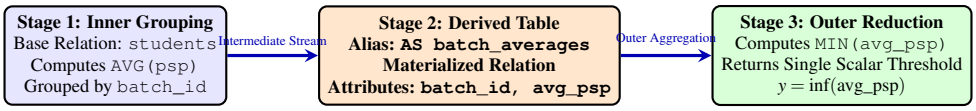



Figure 2: Derived Table Resolution for Multi-Stage Aggregation. SQL's prohibition against nested aggregates (`MIN(AVG())`) is resolved by materializing the first-stage group averages into a derived table (`batch_averages`) upon which the second-stage outer reduction operates.

- **Existential Bound (`ANY`)**: *"Virat has scored more than any of the other players."* Because Virat's score (80) exceeds at least one player (e.g., Kuldeep with 70, Bumrah with 78), the existential condition holds (TRUE).

#### 5.3.1. Industrial Query Formulations

```
-- 1. Students whose PSP >= minimum PSP of EVERY batch:
SELECT * FROM students
WHERE psp >= ALL (
    SELECT min(psp)
    FROM students
    GROUP BY batch_id
);

-- 2. Students whose PSP > minimum PSP of AT LEAST ONE
    batch:
SELECT * FROM students
WHERE psp > ANY (
    SELECT min(psp)
    FROM students
    GROUP BY batch_id
);
```

### 5.4. Vacuous Truth and Three-Valued Logic Hazards

Table 2 exposes critical boundary conditions in SQL quantified comparisons:

1. **Vacuous Truth over Empty Sets**: If the inner subquery produces zero rows ($S = \emptyset$), first-order logic dictates that universal statements are vacuously true:

$$x > \text{ALL}(\emptyset) \equiv \text{TRUE} \quad \text{for any value of } x \tag{10}$$

   Conversely, existential quantification over an empty set is always false:

$$x > \text{ANY}(\emptyset) \equiv \text{FALSE} \tag{11}$$

2. **NULL Contamination in ALL**: If the set $S$ contains even a single `NULL` value, evaluating $x > \text{ALL}(S)$ can never return True unless $x > s$ holds for all known values and the unknown value cannot violate it. In practice, comparing against sets with nulls frequently yields `UNKNOWN`, causing `WHERE` clauses to silently discard qualifying records.

## 6. Correlated Subqueries and Computational Complexity

An independent subquery evaluates once, detached from the outer query context. In contrast, a **Correlated Subquery** references attributes from the enclosing query block, creating an interdependent execution loop.

### 6.1. Operational Definition and Dependency Binding

**Definition 2** (Correlated Subquery)**.** *A subquery $Q_{inner}$ is correlated with an enclosing query $Q_{outer}$ if the evaluation of $Q_{inner}$ requires parameter bindings $t_{outer}[A]$ from the active tuple of $Q_{outer}$:*

$$Q_{inner} = Q_{inner}(t_{outer}[A]) \tag{12}$$

*Because $Q_{inner}$ is parameterized by outer tuple attributes, it cannot be pre-computed as a static standalone relation.*

### 6.2. Case Study: Cohort-Relative Thresholds

Consider the classic enterprise query: *"Find all students whose PSP is strictly greater than the average PSP of their own respective batch."*

```
SELECT s1.id, s1.name, s1.batch_id, s1.psp
FROM students s1
WHERE s1.psp > (
    SELECT AVG(s2.psp)
    FROM students s2
    WHERE s2.batch_id = s1.batch_id -- Correlation
     Binding!
);
```

### 6.3. Computational Complexity Trade-Offs

In a naive execution engine, the procedural execution of this query behaves as a nested loop:

```
FOR each row s1 IN students DO:
    current_batch = s1.batch_id
    current_psp   = s1.psp

    -- Inner query re-executes for EVERY outer row:
    batch_avg = EXECUTE(
        SELECT AVG(psp) FROM students WHERE batch_id =
     current_batch
    )

    IF current_psp > batch_avg THEN:
        EMIT s1
```

**Theorem 3** (Correlated Subquery Nested Loop Complexity)**.** *Let $N$ denote the cardinality of the outer relation and $M$ denote the cardinality of the inner relation. In the absence of index-assisted lookup or caching, unoptimized correlated subquery execution requires:*

$$T_{correlated} = \mathcal{O}(N \cdot M) \tag{13}$$

*For a relation of $10^6$ rows, an unoptimized correlated subquery forces $10^{12}$ tuple comparisons, causing severe CPU thread locking and query timeouts.*

Table 2: Complete Truth and Operational Semantics of Quantified Comparison Predicates

| **Predicate Expression** | **Equivalent Scalar Formulation** | **Truth Condition** | **Empty Set Result ($S = \emptyset$)** | **NULL Contamination Hazard** |
|---|---|---|---|---|
| $x > \text{ALL}(S)$ | $x > \max(S)$ | True iff $x > s$ for every $s \in S$ | **TRUE** (Vacuous Truth) | If any $s = \text{NULL}$ and $x \leq \max$, yields `UNKNOWN`. |
| $x < \text{ALL}(S)$ | $x < \min(S)$ | True iff $x < s$ for every $s \in S$ | **TRUE** (Vacuous Truth) | If any $s = \text{NULL}$ and $x \geq \min$, yields `UNKNOWN`. |
| $x = \text{ALL}(S)$ | $x = s_1 \wedge \lvert S \rvert = 1$ | True iff all elements equal $x$ | **TRUE** (Vacuous Truth) | Yields `UNKNOWN` if any element is `NULL`. |
| $x > \text{ANY}(S)$ | $x > \min(S)$ | True iff $x > s$ for at least one $s$ | **FALSE** | If no element satisfies $x > s$ and `NULL` exists $\rightarrow$ `UNKNOWN`. |
| $x < \text{ANY}(S)$ | $x < \max(S)$ | True iff $x < s$ for at least one $s$ | **FALSE** | If no element satisfies $x < s$ and `NULL` exists $\rightarrow$ `UNKNOWN`. |
| $x = \text{ANY}(S)$ | $x$ IN $S$ | True iff $x$ equals at least one $s$ | **FALSE** | If $x \notin S$ and `NULL` exists $\rightarrow$ `UNKNOWN`. |

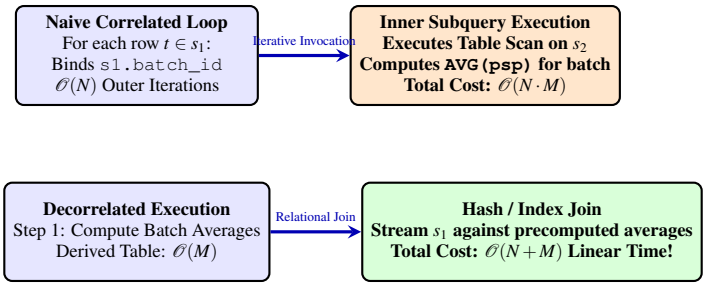



Figure 3: Correlated Execution Dynamics vs. Optimizer Subquery Decorrelation. Naive correlated execution incurs quadratic $\mathcal{O}(N \cdot M)$ nested loop latency. Modern optimizers decorrelate the query into a pre-aggregated hash join, reducing execution to linear $\mathcal{O}(N+M)$ time.

### 6.4. Optimizer Decorrelation and Subquery Unnesting

To eliminate the quadratic performance penalty, modern query optimizers deploy **Subquery Decorrelation** (also termed Subquery Unnesting). The optimizer rewrites the correlated subquery into an explicit Inner Join against a pre-aggregated derived table:

```
-- Decorrelated Relational Equivalence Plan:
SELECT s.id, s.name, s.batch_id, s.psp
FROM students s
INNER JOIN (
    SELECT batch_id, AVG(psp) AS avg_batch_psp
    FROM students
    GROUP BY batch_id
) AS b_avg ON s.batch_id = b_avg.batch_id
WHERE s.psp > b_avg.avg_batch_psp;
```

As illustrated in Figure 3, the decorrelated query executes in linear $\mathcal{O}(N+M)$ time: the batch averages are computed once in a single aggregation pass and probed via in-memory hash join.

## 7. Existence Quantification: EXISTS vs. IN and NULL-Safety

The `EXISTS` operator introduces existential quantification over relational subqueries.

### 7.1. Operational Semantics of EXISTS

**Definition 4** (Existential Predicate Evaluation)**.** *Given a subquery $Q$, the predicate $EXISTS(Q)$ evaluates to a strict two-valued Boolean:*

$$EXISTS(Q) = \begin{cases} TRUE & \text{if } |Q| \geq 1 \\ FALSE & \text{if } |Q| = 0 \end{cases} \tag{14}$$

Unlike scalar and columnar subqueries, `EXISTS` does not consume or project data values; it tests purely whether the candidate tuple set is non-empty. Consequently, the projection list inside `EXISTS` is completely ignored by the query planner:

```
-- Syntactically identical to the engine:
SELECT * FROM students s WHERE EXISTS (SELECT 1 FROM
    ...);
SELECT * FROM students s WHERE EXISTS (SELECT * FROM
    ...);
SELECT * FROM students s WHERE EXISTS (SELECT 1/0 FROM
    ...); -- No divide-by-zero error!
```

#### 7.1.1. Short-Circuit Optimization

Because `EXISTS` only requires proof of at least one row ($|Q| \geq 1$), the storage engine executes **Short-Circuit Optimization**: scanning terminates immediately upon encountering the first qualifying index entry, avoiding full table scans.

### 7.2. The Fatal NOT IN Null Trap vs. NOT EXISTS Immunity

The most profound architectural distinction between `IN` and `EXISTS` occurs during negative filtering:

Table 3: Architectural Comparison: IN vs. EXISTS vs. INNER JOIN

| Construct | Evaluation Paradigm | NULL-Handling Behavior | Short-Circuiting Capability | Duplicate Cardinality Impact |
|---|---|---|---|---|
| `IN (Subquery)` | Set-membership ($\in$) | Contaminated by `NULL` in subquery; `NOT IN` drops 100% rows. | Requires full subquery materialization or semi-join. | Preserves outer cardinality; duplicates ignored. |
| `EXISTS (Subquery)` | Existential test ($\|Q\| \geq 1$) | **Immune to NULLs**; operates strictly in $\{\text{TRUE}, \text{FALSE}\}$. | **Immediate short-circuit** on first matching row. | Preserves outer cardinality; duplicate matches irrelevant. |
| `INNER JOIN` | Cartesian match ($\bowtie_\theta$) | Discards null keys under 3VL ($NULL = NULL \rightarrow$ UNKNOWN). | None; evaluates complete join space. | **Multiplies outer rows** if multiple inner matches exist! |

- **The Fatal `NOT IN` Null Trap**: Suppose a subquery returns a set containing a single `NULL`: $\{1, 2, \text{NULL}\}$. Under De Morgan's laws:

$$x \text{ NOT IN } \{1, 2, \text{NULL}\} \iff (x \neq 1) \wedge (x \neq 2) \wedge (x \neq \text{NULL}) \tag{15}$$

  Because $(x \neq \text{NULL})$ evaluates to **`UNKNOWN`** under Three-Valued Logic (3VL), the entire conjunction evaluates to **`UNKNOWN`** for every row in the database. The query returns **zero rows**, silently losing all data!

- **The Absolute Immunity of `NOT EXISTS`**: The `NOT EXISTS` operator tests row count, not value equality:

$$\text{NOT EXISTS}(Q) \iff |Q| = 0 \tag{16}$$

  If the subquery produces a row where every attribute is `NULL`, that row still physically exists ($|Q| = 1$). Therefore, `NOT EXISTS` evaluates strictly to **`FALSE`** in two-valued logic, completely immune to NULL data poisoning.

# 8. Relational Views: Virtual Table Architecture and Updatability

In enterprise database systems, a **View** is a stored query specification persisted in the database data dictionary that presents itself to client applications as a virtual relation.

## 8.1. Architectural Foundations of Views

As formal architectural abstractions, views provide five enterprise benefits:

1. **Virtual Storage Economy**: Views store zero persistent data rows on disk. Only the parsed SQL query definition and metadata are recorded in the data dictionary.

2. **Live Dynamic Materialization**: Whenever an application queries a view, the database engine resolves the underlying base relations dynamically, guaranteeing immediate consistency without cache staleness.

3. **Query Complexity Encapsulation**: Multi-table joins, window functions, and multi-stage subqueries are encapsulated behind clean relational interfaces:

```
CREATE VIEW active_student_roster AS
SELECT s.id, s.name, b.name AS batch_name, s.
 psp
FROM students s
INNER JOIN batches b ON s.batch_id = b.id
WHERE s.psp >= 70.0;

-- Client query executes seamlessly:
SELECT * FROM active_student_roster WHERE
 batch_name = 'Alpha';
```

4. **Security and Granular Access Masking**: Database administrators grant permissions on the view while revoking base table access. Sensitive columns (e.g., social security numbers, password hashes) and unauthorized rows (e.g., other tenants' records) are strictly hidden.

5. **Schema Decoupling**: Physical database tables can be refactored or normalized without altering public view contracts, shielding downstream microservices from breaking schema changes.

## 8.2. Physical Execution: AST Merge vs. Temporary Table Materialization

When a client executes a query against a view, the query processor employs one of two physical execution algorithms:

1. **`ALGORITHM = MERGE`**: The query optimizer unpacks the view definition and merges its `WHERE` clauses, joins, and projections directly into the referencing query's Abstract Syntax Tree.
   - Predicate pushdown operates across the combined query, allowing outer filters (`WHERE psp > 90`) to utilize the base table's B+ Tree indexes directly.
   - Memory overhead is negligible ($\mathcal{O}(1)$).

2. **`ALGORITHM = TEMPTABLE`**: When a view contains non-mergeable constructs (such as `GROUP BY`, aggregate functions, `DISTINCT`, `HAVING`, `UNION`, or window functions), the engine cannot fuse the ASTs.

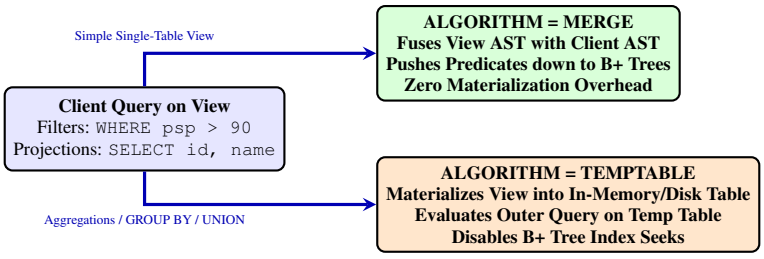



Figure 4: Relational View Execution Strategies in MySQL: AST Merging vs. Temporary Table Materialization. View merging fuses client predicates directly into the underlying B+ Tree scans, whereas temporary table materialization computes the full view before evaluating outer filters.

- The engine materializes the entire view into an internal in-memory or on-disk temporary table.
- The outer query then scans this temporary table. Outer filters cannot be pushed down into base table indexes, dramatically increasing query latency.

### 8.3. View Updatability and Determinism

A common enterprise requirement is executing Data Manipulation Language mutations (`INSERT`, `UPDATE`, `DELETE`) through view interfaces.

**Theorem 5** (View Updatability Invariant)**.** *A relational view is updatable if and only if there exists an injective (one-to-one) deterministic mapping between each row in the view and exactly one physical tuple in an underlying base table.*

Consequently, views that contain the following operations are strictly **read-only**:

- Aggregate reduction functions (`SUM`, `COUNT`, `AVG`, etc.).
- Relational grouping (`GROUP BY`) and post-filtering (`HAVING`).
- Deduplication operators (`DISTINCT`).
- Set operations (`UNION`, `INTERSECT`, `EXCEPT`).
- Multi-table joins where the update target is not an unambiguous key-preserved table.

## 9. The Comprehensive SQL DQL Logical Query Lifecycle

The complete logical execution lifecycle of a complex relational query containing subqueries and views formalizes why clauses behave as observed:

Understanding this lifecycle explains why:

1. Derived tables in `FROM` (Step 2) are evaluated before `WHERE` (Step 4), allowing derived aggregates to act as static filters for the outer query.
2. Correlated subqueries in `WHERE` (Step 4) are invoked once per surviving outer tuple, whereas decorrelated joins execute once at Step 2.
3. Aliases created in `SELECT` (Step 8) cannot be referenced in `WHERE` (Step 4) or `GROUP BY` (Step 5).

## 10. Conclusion and Enterprise Engineering Blueprint

Relational subqueries and virtual views represent foundational abstractions for modular, secure, and expressive query engineering. This research paper has formalized the core mechanics governing subqueries, quantified comparisons, and view pipelines:

1. **Enforce Scalar Subquery Determinism**: Guard scalar subqueries with unique indexed keys or aggregate reduction functions (`MAX`, `MIN`) to eliminate runtime Error 1242 cardinality violations.
2. **Mandate Derived Table Aliases**: Always assign explicit aliases to table expressions in the `FROM` clause to satisfy ANSI SQL data dictionary requirements.
3. **Resolve Nested Aggregates via Derived Tables**: Never attempt direct aggregate nesting (`MIN(AVG())`); encapsulate the inner grouping in a derived table and compute outer summaries over the materialized result.
4. **Prefer NOT EXISTS over NOT IN**: Always deploy `NOT EXISTS` for negative set filtering to guarantee complete immunity against the fatal Three-Valued Logic NULL contamination trap.
5. **Beware of Vacuous Truth in ALL**: Remember that universal quantification against an empty subquery ($x > \text{ALL}(\emptyset)$) evaluates vacuously to `TRUE`.
6. **Deploy ANY for Quantified Inequalities**: Use `ANY` when testing relational inequalities ($>, <, \geq, \leq$) against multi-row columnar subqueries, as `IN` is restricted strictly to equality comparisons.

Table 4: Comprehensive SQL DQL Logical Query Execution Pipeline with Nested Subqueries and Views

| Step | SQL Processing Phase | Relational Operation | Subquery / View Resolution Mechanism |
|---|---|---|---|
| 1 | **`VIEW EXPANSION`** | Query Tree Rewriting | Unpacks views; fuses AST via `MERGE` or schedules `TEMPTABLE` materialization. |
| 2 | **`FROM / JOIN`** | Base Cartesian Product / Joins | Materializes derived tables; executes join algorithms (INLJ, Hash Join, Sort-Merge). |
| 3 | **`ON`** | Join Predicate Evaluation | Evaluates join predicates; governs outer join preservation. |
| 4 | **`WHERE`** | Tuple Selection ($\sigma$) | **Executes scalar, `IN`, `ALL`, `ANY`, and `EXISTS` subqueries; applies SARGable index seeks.** |
| 5 | **`GROUP BY`** | Equivalence Partitioning | Groups tuples into equivalence buckets; collates NULLs into single group. |
| 6 | **`AGGREGATION`** | Multi-Set Reduction ($f$) | Computes `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`; eliminates NULL values. |
| 7 | **`HAVING`** | Group Selection ($\sigma_{\text{group}}$) | **Evaluates group-level subqueries and aggregate thresholds post-aggregation.** |
| 8 | **`SELECT`** | Projection ($\pi$) & Expressions | Evaluates scalar subqueries in projection list; formats aliases and string concatenations. |
| 9 | **`DISTINCT`** | Duplicate Tuple Suppression | Deduplicates tuples via streaming B+ Tree scans or in-memory hash sets. |
| 10 | **`ORDER BY`** | Physical Sequence Ordering | Filesort buffer allocation; multi-column deterministic tie-breaking. |
| 11 | **`LIMIT / OFFSET`** | Stream Windowing | Discards offset records; emits final output window. |

7. **Decorrelate Quadratic Loops**: Refactor correlated subqueries into explicit derived table joins to reduce execution latency from $\mathcal{O}(N \cdot M)$ to $\mathcal{O}(N+M)$.

8. **Design Views for AST Merging**: Maintain simple, unaggregated view definitions where possible to allow the query planner to deploy `ALGORITHM = MERGE`, preserving B+ Tree index range seeks.

9. **Isolate Sensitive Data via Views**: Restrict application database credentials to virtual views with masked columns and row-level tenant filters, revoking direct base table permissions.

10. **Respect View Updatability Boundaries**: Never issue `UPDATE` or `INSERT` statements through views that contain aggregations, `GROUP BY`, or `DISTINCT`.

By adhering to this architectural blueprint, database engineers and software architects guarantee deterministic query correctness, bounded computational complexity, and secure multi-tenant data isolation across high-scale enterprise systems.

## References

[1] E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[2] C. J. Date, *Database in Depth: Relational Theory for Practitioners*. Sebastopol, CA: O'Reilly Media, 2005.

[3] Scaler Academy, "SQL Class 6: Subqueries, Correlated Subqueries, Quantified Comparisons, and Relational Views," Technical Lecture Notes (Note 22 Dec 2025), Nov. 2026.

[4] Oracle Corporation, "MySQL 8.4 Reference Manual: Subqueries, Derived Tables, and Views Optimization," Oracle Technology Network, 2024. [Online]. Available: `https://dev.mysql.com/doc/refman/8.4/en/subqueries.html`

[5] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, "Access Path Selection in a Relational Database Management System," in *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*, pp. 23–34, 1979.

[6] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Mateo, CA: Morgan Kaufmann Publishers, 1993.